

EcoRuta: Documentación del proyecto

Reyner Rojas Gutiérrez

Makin Artavia Zúñiga

Cipriano Rivera Escobar

Mc Kenneth Frauli Arce

Universidad Nacional de Costa Rica, Campus Coto

41549: Inteligencia Artificial

Ing. Jeremy Elizondo Castro

12 de junio de 2026

Introducción

EcoRuta es una aplicación móvil orientada a personas que desean descubrir rutas para caminar, correr o andar en bicicleta de una manera más personalizada. El problema principal que busca resolver es que, en muchos casos, las aplicaciones de mapas muestran trayectos genéricos, pero no consideran con suficiente detalle el perfil del usuario, su nivel de actividad, la dificultad de la ruta ni sus preferencias previas. Como resultado, una misma ruta puede ser adecuada para una persona con buen ritmo físico, pero poco conveniente para alguien que apenas comienza o que desea una opción más segura y controlada.

A partir de esa necesidad, el proyecto integra técnicas de inteligencia artificial y algoritmos de búsqueda para apoyar tres decisiones importantes dentro de la experiencia del usuario. La primera es la lectura general del perfil de salud mediante inferencias sencillas basadas en datos corporales y de actividad. La segunda es el cálculo de trayectos con el algoritmo A*, que permite encontrar caminos útiles dentro del grafo de rutas obtenido desde OpenStreetMap. La tercera es la recomendación personalizada de rutas públicas mediante un modelo de redes neuronales tipo two-tower, que compara características del usuario con características de las rutas disponibles.

La selección de estas técnicas no fue casual. Se eligió un motor de inferencia local porque era una opción comprensible, ligera y fácil de integrar dentro del perfil del usuario sin depender de un modelo médico complejo. Se eligió A* porque es un algoritmo ampliamente conocido para encontrar rutas eficientes cuando se tiene un grafo y una heurística de distancia. Finalmente, se eligió un enfoque two-tower porque ofrece una forma práctica de aprender afinidades entre usuarios y rutas, incluso cuando el número de registros disponibles aún es pequeño y el sistema necesita escalar en el futuro.

El objetivo de este documento es describir el proyecto EcoRuta desde una perspectiva técnica y funcional, con énfasis especial en la parte de inteligencia artificial. Se presenta el sustento teórico de las técnicas utilizadas, la arquitectura general del sistema, fragmentos de implementación, resultados obtenidos en pruebas iniciales y un manual de usuario para su instalación y ejecución.

Marco teórico

Inteligencia artificial aplicada al proyecto

En EcoRuta, la inteligencia artificial no aparece como una sola pieza aislada, sino como un conjunto de decisiones que permiten personalizar la experiencia. En términos simples, la IA se usa para interpretar datos del usuario, apoyar la selección de rutas y ordenar recomendaciones. Esto significa que la aplicación no se limita a mostrar información, sino que intenta producir una respuesta adaptada al contexto de cada persona.

Desde esta perspectiva, EcoRuta combina dos enfoques distintos. Por un lado, utiliza inferencia basada en reglas para producir indicadores orientativos de salud. Por otro lado, usa aprendizaje profundo para el sistema de recomendación. Esta mezcla resulta conveniente porque no todas las partes del sistema necesitan la misma complejidad: algunas decisiones pueden resolverse con reglas claras y otras se benefician más de aprender patrones desde los datos.

Inferencia basada en reglas para el perfil de salud

Los sistemas basados en reglas pertenecen a una línea clásica de la inteligencia artificial conocida como sistemas expertos. De acuerdo con NIST, un sistema experto separa el conocimiento del dominio y el mecanismo de inferencia, de manera que las reglas representan hechos o heurísticas y el motor de inferencia las aplica para obtener conclusiones. Llevado a un

lenguaje más sencillo, esto significa que el sistema toma datos de entrada, revisa condiciones definidas previamente y devuelve una interpretación consistente con esas reglas.

En EcoRuta, este enfoque aparece en el módulo de salud. El sistema no intenta diagnosticar enfermedades ni reemplazar una valoración profesional. Lo que hace es construir una lectura general a partir de datos como peso, altura, fecha de nacimiento, rutas por semana, kilómetros, minutos de actividad y consistencia reciente. A partir de eso, calcula un IMC, clasifica el nivel de actividad y genera un wellness score orientativo.

El cálculo del IMC se apoya en la fórmula peso entre altura al cuadrado. La Organización Mundial de la Salud indica que, en adultos, un IMC igual o superior a 25 se considera sobrepeso y un IMC igual o superior a 30 se considera obesidad. EcoRuta toma esa lógica como una referencia inicial y la simplifica en cuatro categorías: bajo peso, normal, sobrepeso y obesidad. Esta clasificación no pretende ser un juicio clínico final, sino un indicador útil para mostrar contexto al usuario.

Además del IMC, el módulo valora la actividad semanal. La OMS también recomienda que los adultos realicen al menos 150 minutos semanales de actividad física moderada, y destaca que hacer algo de actividad es mejor que no hacer nada. EcoRuta traduce esa idea a una escala sencilla por rutas, kilómetros y minutos por semana. Luego promedia esas señales para ubicar al usuario en niveles como baja, moderada, buena o alta actividad. Esta salida es especialmente útil porque le da significado a datos que, de otro modo, estarían dispersos.

La ventaja de este enfoque es su claridad. Las reglas se pueden revisar, ajustar y explicar con facilidad. También es una solución ligera para una aplicación móvil, ya que no exige entrenamiento continuo ni procesamiento intensivo. Su principal limitación es que depende del

criterio con que se definan los umbrales y no aprende automáticamente de nuevos comportamientos. Aun así, para una primera capa de interpretación del perfil, resulta apropiado y coherente con el alcance del proyecto.

Algoritmo A* para cálculo de rutas

El problema de encontrar una ruta entre dos puntos puede modelarse como un grafo, donde los nodos representan ubicaciones y las aristas representan conexiones transitables. Sobre este tipo de estructura, uno de los algoritmos más conocidos es A*. Hart, Nilsson y Raphael lo formalizaron en 1968 como un método heurístico para determinar caminos de costo mínimo. Su idea central es combinar el costo ya recorrido con una estimación de la distancia restante hacia la meta.

En palabras más simples, A* no avanza a ciegas. En cada paso compara opciones y prioriza las que parecen acercarse mejor al destino. Para hacerlo, utiliza una función de evaluación que normalmente se expresa como $f(n) = g(n) + h(n)$, donde $g(n)$ representa el costo acumulado y $h(n)$ representa la heurística. Si la heurística está bien elegida, el algoritmo reduce exploraciones innecesarias y encuentra caminos útiles con buena eficiencia.

En EcoRuta se utiliza A* sobre un grafo construido a partir de datos de OpenStreetMap procesados por Overpass. Cada arista tiene como costo principal la distancia en metros, mientras que la heurística se calcula con la distancia haversine entre coordenadas geográficas. Esto permite que la búsqueda tome en cuenta la forma real del espacio geodésico y no solo una aproximación plana.

El proyecto también agrega decisiones prácticas alrededor de A*. Antes de rutear, filtra la componente conexa principal del grafo para reducir errores por segmentos desconectados.

Después selecciona nodos ancla cercanos al punto de inicio y al punto de destino, y ensaya varias combinaciones hasta encontrar una conexión válida. Para la variante más desafiante, en lugar de redefinir el costo base de A^* , la aplicación calcula una ruta más corta hacia un posible pico de elevación y luego otra hacia el destino, quedándose con la opción que gana más desnivel positivo.

Este uso de A^* es adecuado para EcoRuta porque mantiene un equilibrio entre simplicidad, velocidad y capacidad de explicación. Si se quisiera avanzar en el futuro, se podrían explorar costos multiobjetivo, penalizaciones por superficie o seguridad vial e incluso técnicas de replaneamiento dinámico cuando cambien las condiciones del entorno.

Redes neuronales two-tower para recomendación

Los sistemas de recomendación buscan responder una pregunta sencilla pero importante: qué contenido tiene más sentido mostrar a cada usuario. En EcoRuta, el contenido recomendado son rutas públicas creadas por otros usuarios. La dificultad está en que no basta con ver la ruta de forma aislada; también hace falta entender quién es la persona que la va a recibir, qué actividad le gusta, en qué región suele moverse y qué tipo de recorridos ha guardado antes.

Para resolver esto, el proyecto usa una arquitectura de redes neuronales conocida como two-tower o dual encoder. En este enfoque existe una torre para representar al usuario y otra torre para representar a la ruta. El objetivo no es memorizar una lista fija de coincidencias, sino aprender representaciones densas, llamadas embeddings, que faciliten comparar afinidad entre ambas partes.

El trabajo de Covington, Adams y Sargin sobre recomendaciones de YouTube explica un enfoque de dos etapas donde una red genera candidatos y otra red ayuda al ranking posterior. En

ese contexto, los autores destacan que la red de generación de candidatos usa el historial y el contexto del usuario para producir embeddings útiles para discriminar entre elementos de un gran catálogo EcoRuta retoma esa idea a menor escala: crea una representación del usuario y otra de la ruta, y luego las compara mediante producto punto para producir un score.

En la torre de usuario de EcoRuta se incluyen variables como actividad favorita, región, edad aproximada, kilómetros acumulados, rutas completadas y estadísticas derivadas de las rutas públicas guardadas. En la torre de ruta se usan actividades de la ruta, región, distancia, duración y elevación. Las variables categóricas se convierten en embeddings y las numéricas se normalizan con funciones logarítmicas o escalas simples. Luego, cada torre pasa por capas densas con activación ReLU y termina en un vector de proyección.

Para entrenar el modelo, se toman como ejemplos positivos las rutas públicas que el usuario realmente guardó. Luego se generan ejemplos negativos a partir de rutas públicas que ese mismo usuario no guardó. Este esquema permite aproximar una tarea de preferencia implícita: si una ruta fue guardada, probablemente tenía interés para ese perfil; si no fue guardada, se asume una afinidad menor. El entrenamiento se hace con BCEWithLogitsLoss y optimizador Adam.

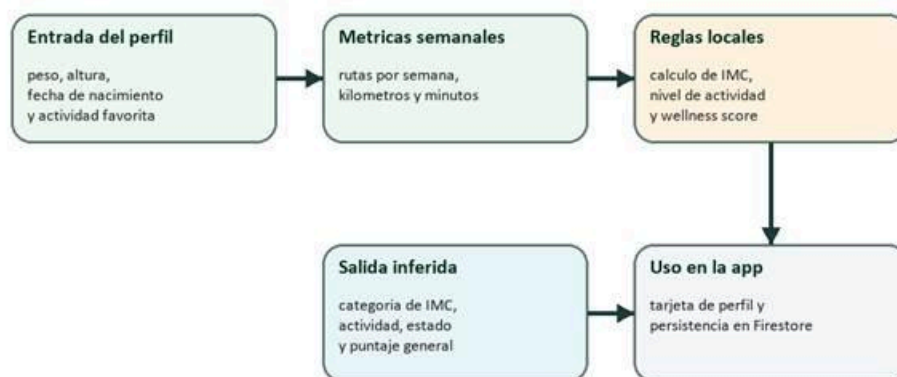
Además del entrenamiento local del modelo, se implementó un proceso de despliegue y actualización automática para facilitar el funcionamiento del sistema de recomendaciones dentro de EcoRuta. El modelo y el backend de recomendaciones fueron empaquetados mediante Docker y desplegados en la plataforma Render, permitiendo exponer el sistema como una API accesible desde la aplicación móvil. De esta forma, la app puede consultar recomendaciones en tiempo real mediante solicitudes HTTP al servicio desplegado.

Para mantener actualizado el modelo, se configuró un flujo automatizado utilizando GitHub Actions. Este proceso ejecuta diariamente el entrenamiento de la red two-tower, permitiendo recalcular embeddings y ajustar el modelo conforme se agregan nuevas rutas públicas y nuevas interacciones de los usuarios. Una vez finalizado el entrenamiento, los archivos generados se sincronizan con el servicio desplegado, logrando un esquema básico de actualización continua del sistema de recomendación.

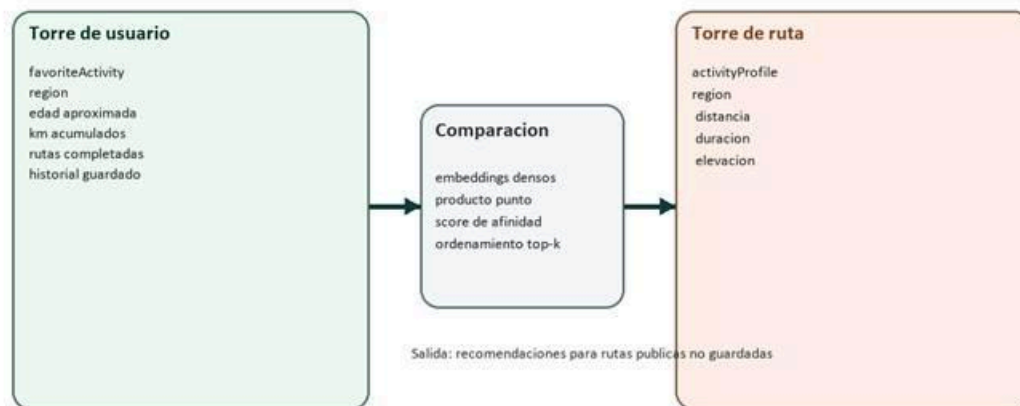
La mayor fortaleza de este enfoque es que separa claramente la representación del usuario y la representación de la ruta, lo cual facilita escalar y reutilizar el modelo. Su principal debilidad actual es el tamaño pequeño del conjunto de entrenamiento. Por eso, en el estado actual del proyecto, las recomendaciones deben interpretarse como una primera aproximación funcional y no como un sistema definitivo de alta precisión.

Figura 1.

Flujo de inferencia de salud en EcoRuta



Nota. Flujo simplificado del módulo de inferencia de salud.

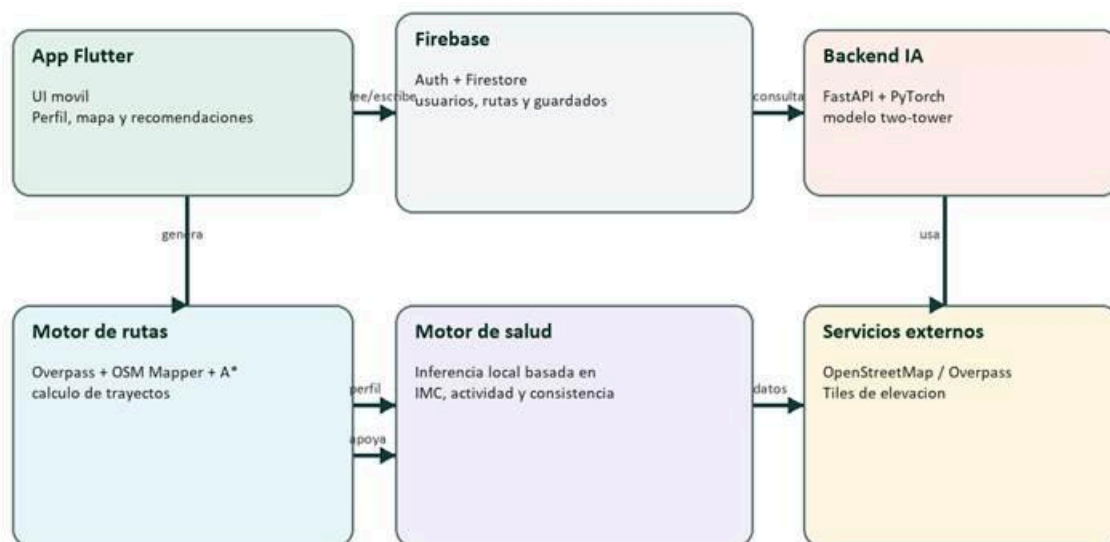
Figura 2.*Esquema del recomendador*

Nota. Esquema conceptual del recomendador two-tower usado en EcoRuta.

Diseño del sistema

EcoRuta fue diseñado como un sistema modular donde la aplicación móvil se encarga de la experiencia del usuario y varios servicios auxiliares se ocupan de los datos, la inferencia y la recomendación. La capa principal está construida con Flutter, lo que permite mantener una sola base de código para varias plataformas. La autenticación y el almacenamiento principal se apoyan en Firebase, mientras que la información geoespacial se obtiene a partir de OpenStreetMap mediante consultas a Overpass.

La arquitectura general puede entenderse en seis módulos: interfaz móvil, gestión de usuarios, motor de salud, motor de rutas, backend de recomendaciones y servicios externos. Esta separación favorece la mantenibilidad del proyecto, porque cada módulo tiene una responsabilidad clara y un punto de integración visible.

Figura 3.*Arquitectura general*

Nota. Arquitectura general del sistema EcoRuta.

Tabla 1.*Principales módulos de la aplicación*

Módulo	Responsabilidad principal	Archivos relacionados
Perfil de usuario	Gestiona datos personales, progreso e indicadores de salud.	profile_screen.dart, user_model.dart, health_inference.dart
Exploración y rutas	Consulta datos de OSM, arma el grafo y calcula trayectos.	explore_provider.dart,

		overpass_service.dart t, a_star_router.dart
Recomendaciones	Consulta la API de IA y muestra rutas sugeridas.	recommendations_service.dart, recommendations_table.dart
Backend IA	Entrena el modelo two-tower y expone endpoints FastAPI.	train_two_tower.py, predict_api.py
Persistencia	Administra autenticación y almacenamiento en Firestore.	auth_service.dart, saved_routes_service.dart

Nota. Esta tabla muestra los principales módulos de la aplicación EcoRuta para la integración de los algoritmos de Inteligencia Artificial.

Flujo de generación de rutas

1. El usuario selecciona origen, destino y tipo de actividad.
2. La app calcula un bounding box alrededor de los puntos seleccionados.
3. Overpass devuelve nodos y vías candidatas desde OpenStreetMap.
4. El mapper convierte esa respuesta en un grafo con nodos y aristas.
5. Se filtra la componente conexa principal para evitar segmentos aislados.
6. Se buscan nodos ancla cercanos y se ejecuta A* para la ruta más corta.
7. Si el usuario desea una ruta más desafiante, se evalúa una variante con elevación.

8. La ruta final se presenta con distancia, duración estimada y desnivel positivo.

Flujo de recomendaciones

9. La app obtiene el UID del usuario autenticado.
10. El servicio Flutter llama al endpoint `/recommendations/{userId}`.
11. FastAPI carga usuarios, rutas públicas y documentos de `saved_public_routes` desde Firestore.
12. El backend arma las características del usuario y las rutas.
13. El modelo two-tower calcula embeddings y scores de afinidad.
14. Se excluyen rutas propias y rutas ya guardadas por el usuario.
15. La API devuelve un top-k ordenado por score.
16. La interfaz muestra las rutas sugeridas y permite abrirlas o guardarlas.

Implementación

La implementación combina lógica en Dart para la aplicación móvil y Python para el backend de entrenamiento e inferencia. A continuación se muestran algunos fragmentos representativos del proyecto.

Fragmento 1. Inferencia de salud

En el archivo de inferencia local se observa que el sistema calcula IMC, clasifica actividad y produce un wellness score a partir de reglas sencillas:

```
final bmi = _calculateBmi(input.weightKg, input.heightCm);
final bmiCategory = _bmiCategoryFor(bmi);
final activityLevel = _activityLevelFor(input);
final wellnessScore = _wellnessScoreFor(
  bmiCategory: bmiCategory,
  activityLevel: activityLevel,
  consistencyScore: input.activityConsistencyScore,
```

```
);
```

Este fragmento resume bien la filosofía del módulo: tomar datos simples del usuario y traducirlos a una salida entendible. No hay un clasificador entrenado, sino reglas definidas por el proyecto.

Fragmento 2. Búsqueda con A*

El motor de rutas mantiene los puntajes acumulados, la heurística y una cola de prioridad para decidir el siguiente nodo a expandir:

```
final gScore = <int, double>{startId: 0.0};
final fScore = <int, double>{
    startId: _heuristic(startNode, goalNode, profile, preference),
};
final openQueue = PriorityQueue<_AStarEntry>((a, b) => a.f.compareTo(b.f))
    ..add(_AStarEntry(startId, fScore[startId]));
```

En la práctica, esto significa que el algoritmo siempre prioriza las rutas candidatas que combinan menor costo acumulado y mejor aproximación al destino. La heurística usada en EcoRuta se basa en la distancia geográfica.

Fragmento 3. Modelo two-tower

En el backend de recomendaciones se construyen dos torres independientes y luego se comparan sus salidas mediante producto punto:

```
self.user_tower = nn.Sequential(
    nn.Linear(user_input_dim, hidden_dim),
    nn.ReLU(),
    nn.Linear(hidden_dim, projection_dim),
)
self.route_tower = nn.Sequential(
    nn.Linear(route_input_dim, hidden_dim),
    nn.ReLU(),
```

```

nn.Linear(hidden_dim, projection_dim),
)

logits = (user_embedding * route_embedding).sum(dim=1)

```

Este bloque muestra la esencia de la recomendación. Cada torre aprende una representación compacta y después ambas se comparan. Si el producto punto es alto, la ruta tiene más afinidad con el usuario.

Decisiones de implementación relevantes

- Las variables categóricas de usuario y ruta se convierten en embeddings para no tratar cada valor como texto crudo.
- Las variables numéricas se normalizan con logaritmos o escalas acotadas, lo que ayuda a estabilizar el entrenamiento.
- Las recomendaciones se sirven por API para no cargar el modelo completo dentro del cliente móvil.
- El módulo de salud se mantiene local, lo cual reduce complejidad y permite respuesta inmediata en el perfil.
- El cálculo de rutas se apoya en datos abiertos de OpenStreetMap para mantener coherencia con el propósito ecológico y exploratorio del proyecto.

Resultados

Las pruebas realizadas sobre EcoRuta permitieron verificar que las tres piezas principales del sistema funcionan de manera integrada: el perfil produce inferencias orientativas, el motor de rutas calcula trayectos sobre datos reales del mapa y el backend de recomendaciones devuelve rutas públicas ordenadas por afinidad. Aunque el proyecto todavía se encuentra en una etapa académica y con un conjunto de datos reducido, ya es posible observar resultados funcionales y medibles.

Pruebas funcionales realizadas

- Prueba de registro y perfil: al crear o actualizar un usuario, el sistema recalcula los indicadores de salud y los guarda en Firestore.
- Prueba de exploración: al seleccionar dos puntos en el mapa, la app consulta Overpass, construye el grafo y genera al menos una ruta válida cuando existe conectividad.
- Prueba de recomendaciones: la pantalla de recomendaciones consulta el backend y muestra sugerencias con porcentaje de match sobre rutas públicas.
- Prueba de filtrado: las recomendaciones incluyen rutas propias del usuario y rutas ya guardadas por ese mismo perfil.

Tabla 2.

Métricas observadas en el entrenamiento del modelo

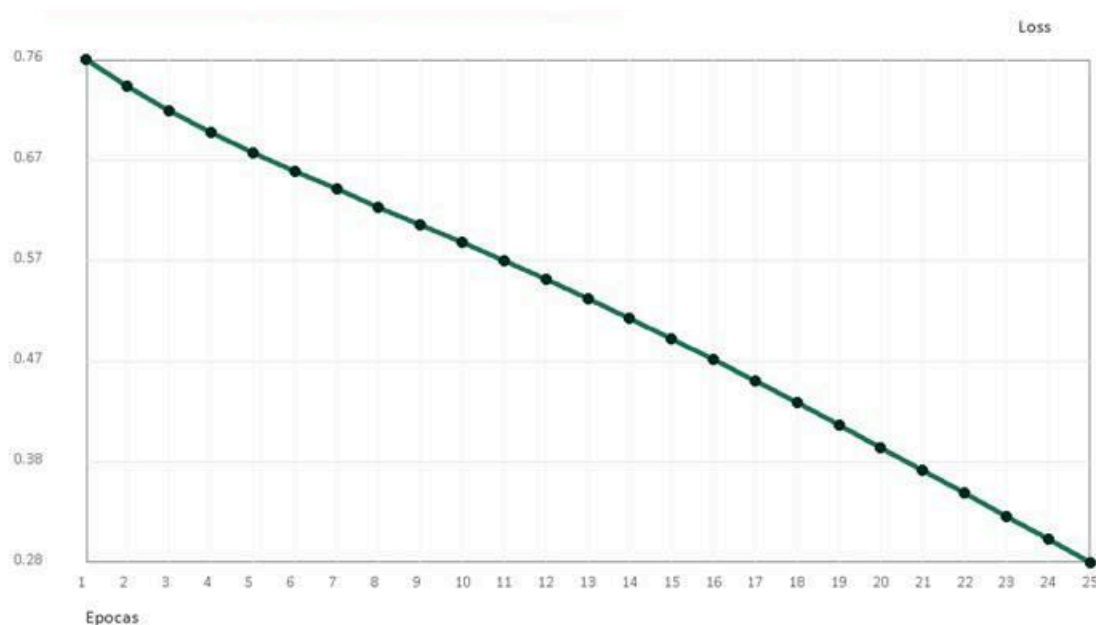
Indicador	Valor observado
Épocas de entrenamiento	25
Ejemplos de entrenamiento	8
Loss inicial	0.7645
Loss final	0.2813
Reducción aproximada del loss	63.21%

Fecha del último	2026-05-28
entrenamiento	

Nota. Esta tabla muestra las métricas observadas durante el desarrollo del entrenamiento del modelo two-tower para la red neuronal.

Figura 4.

Evolución de pérdida



Nota. Historial de pérdida obtenido del entrenamiento guardado en artifacts.

El historial almacenado por el proyecto muestra una disminución progresiva del loss, desde aproximadamente 0.7645 hasta 0.2813 en 25 épocas.

Aunque este comportamiento es positivo, también debe leerse con cautela porque el entrenamiento registrado utiliza un conjunto pequeño de ejemplos. En otras palabras, el modelo

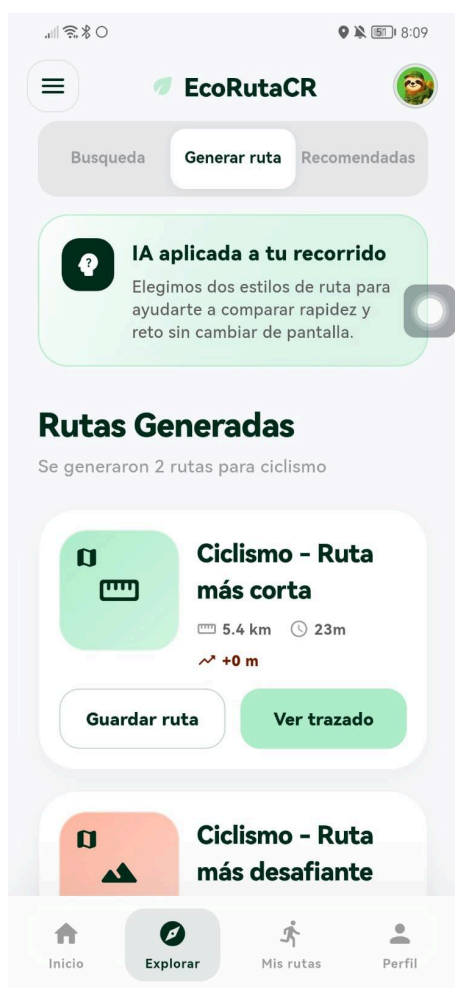
ya muestra capacidad para separar mejor los casos positivos y negativos, pero todavía no existe evidencia suficiente para afirmar que generaliza bien en un entorno más amplio.

Capturas y evidencia visual disponible

Como evidencia visual del proyecto, en el repositorio se dispone de la pantalla de bienvenida y de las vistas desarrolladas para perfil, exploración y recomendaciones. En esta documentación se incluye la imagen disponible del inicio del sistema y se acompaña con diagramas técnicos del flujo interno para reforzar la explicación del funcionamiento.

Figura 5.

Pantalla explorar



Nota. Pantalla de generación de rutas con IA ubicada en la sección explorar de la aplicación.

Figura 6.

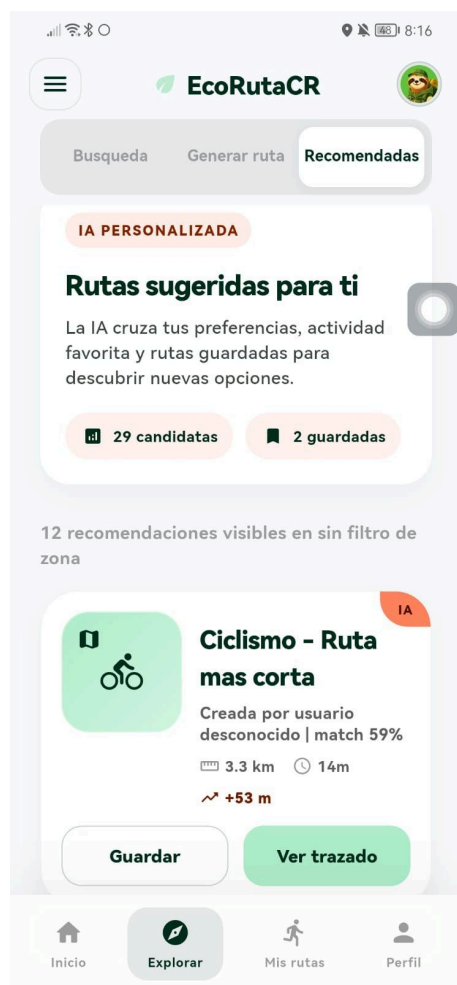
Pantalla de perfil



Nota. Pantalla de indicadores orientativos del perfil.

Figura 7.

Pantalla de recomendaciones



Nota. Pantalla de recomendación con redes neuronales.

Análisis de logros y limitaciones

Entre los logros más importantes se encuentra la integración de varias técnicas en una misma solución: datos geoespaciales reales, cálculo de rutas, inferencia orientativa de salud y recomendaciones personalizadas. También es un acierto que cada módulo pueda explicarse con relativa claridad, algo valioso para fines académicos y de mantenimiento.

La principal limitación es la escala de datos. Los metadatos del modelo muestran solo unos pocos ejemplos de entrenamiento, lo que reduce la robustez del recomendador. Además, el módulo de salud usa reglas fijas, por lo que depende de umbrales definidos manualmente y no

captura todas las variaciones posibles de la condición física. En el caso de A*, la implementación actual privilegia sobre todo la distancia y apenas empieza a incorporar elevación como criterio complementario.

Conclusiones

EcoRuta demuestra que es posible construir una aplicación móvil con funciones de inteligencia artificial sin perder claridad en el diseño. El proyecto integra un motor de inferencia local para el perfil de salud, un algoritmo A* para el cálculo de rutas y un modelo two-tower para recomendaciones de rutas públicas. Estas tres piezas responden a necesidades distintas, pero juntas fortalecen la personalización del sistema.

Desde el punto de vista técnico, la solución resulta valiosa porque mezcla enfoques ligeros y enfoques de aprendizaje profundo de manera razonable. Las reglas del perfil de salud permiten respuestas rápidas y entendibles, mientras que el modelo two-tower abre la puerta a una personalización más rica conforme aumente la cantidad de datos. Por su parte, A* sigue siendo una base sólida para el problema de navegación dentro del grafo.

También queda claro que el proyecto todavía tiene margen de crecimiento. Las recomendaciones podrían mejorar mucho con más interacciones, mejores métricas offline y evaluaciones con usuarios reales. Del mismo modo, el cálculo de rutas podría incorporar criterios de seguridad, superficie, sombra o tráfico, y el módulo de salud podría enriquecerse con históricos más detallados. Aun con esas limitaciones, el estado actual del proyecto ya muestra logros concretos y una dirección técnica coherente.

Manual de usuario

Requisitos generales

- Flutter instalado y configurado para la plataforma de prueba.

- Acceso a Firebase con la configuración incluida en el proyecto.
- Python 3.12 o compatible para el backend de recomendaciones.
- Credencial de service account para consultas administrativas a Firestore.

Instalación y ejecución de la app

17. Abrir la carpeta raíz del proyecto EcoRuta y dirigirse a la carpeta Móvil.
18. Ejecutar la instalación de dependencias con el comando flutter pub get.
19. Verificar que exista una configuración válida de Firebase para la plataforma elegida.
20. Ejecutar la app con flutter run. Si se desea consumir el backend remoto o local de recomendaciones, agregar la variable RECOMMENDATION_API_BASE_URL.

```
flutter pub get
```

```
flutter run --dart-define=RECOMMENDATION_API_BASE_URL=http://127.0.0.1:8000
```

Entrenamiento y API de recomendaciones

21. Instalar dependencias de Python con pip install -r ml_backend/requirements.txt.
22. Definir la variable FIREBASE_SERVICE_ACCOUNT_PATH con la ruta al JSON de service account.
23. Probar conectividad con python ml_backend/train_two_tower.py --smoke-test.
24. Entrenar el modelo con python ml_backend/train_two_tower.py --train --epochs 25 --negative-samples 3.
25. Levantar la API con uvicorn ml_backend.predict_api:app --reload.

```
pip install -r ml_backend/requirements.txt
```

```
$env:FIREBASE_SERVICE_ACCOUNT_PATH="C:\ruta\a\serviceAccountKey.json"
```

```
python ml_backend/train_two_tower.py --smoke-test
```

```
python ml_backend/train_two_tower.py --train --epochs 25 --negative-samples 3
```

```
uvicorn ml_backend.predict_api:app --reload
```

Uso básico dentro del sistema

- Registrar o iniciar sesión para habilitar la persistencia del perfil.
- Completar datos como peso, altura y actividad favorita para mejorar la lectura del perfil.
- Explorar rutas en el mapa seleccionando origen, destino y tipo de actividad.
- Consultar la pestaña de recomendaciones para ver rutas sugeridas por la IA.
- Guardar rutas públicas de interés para enriquecer el historial del usuario.

Posibles mejoras futuras

- Aumentar el conjunto de datos de guardados y rutas públicas para fortalecer el recomendador.
- Incorporar evaluaciones offline como $\text{precision}@k$ o $\text{recall}@k$ cuando exista mayor volumen de datos.
- Agregar nuevas variables al modelo, por ejemplo horario preferido, clima o nivel de dificultad declarado por el usuario.
- Introducir una variante de A^* con costo multiobjetivo para considerar distancia, desnivel y seguridad al mismo tiempo.
- Mejorar la parte visual del documento con capturas reales de perfil y recomendaciones tomadas desde un dispositivo de prueba.

Referencias bibliográficas

- Covington, P., Adams, J., & Sargin, E. (2016). Deep Neural Networks for YouTube Recommendations. Proceedings of the 10th ACM Conference on Recommender Systems.
<https://research.google/pubs/deep-neural-networks-for-youtube-recommendations/>
- Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A Formal Basis for the Heuristic Determination of Minimum Cost Paths. IEEE Transactions on Systems Science and Cybernetics, 4(2), 100-107.
<https://www.cs.auckland.ac.nz/courses/compsci709s2c/resources/Mike.d/astarNilsson.pdf>
- Organización Mundial de la Salud. (2025, 8 de diciembre). Obesity and overweight.
<https://www.who.int/en/news-room/fact-sheets/detail/obesity-and-overweight>
- World Health Organization. (2020). WHO guidelines on physical activity and sedentary behaviour. <https://www.who.int/publications/b/55518>
- National Institute of Standards and Technology. (1982). An overview of expert systems.
<https://doi.org/10.6028/NBS.IR.82-2505>
- Centers for Disease Control and Prevention. (2024). Adult BMI Categories.
<https://www.cdc.gov/bmi/adult-calculator/bmi-categories.html>
- OpenStreetMap contributors. (2026). OpenStreetMap. <https://www.openstreetmap.org/>
- FastAPI. (2026). FastAPI documentation. <https://fastapi.tiangolo.com/>
- PyTorch Contributors. (2026). PyTorch documentation.
<https://pytorch.org/docs/stable/index.html>
- Flutter documentation. (2026). Flutter. <https://docs.flutter.dev/>